

Stress-Testing n-step Soft Actor-Critic under Limited Compute

Краткий проектный отчет по RL final project

Radmir I.

2026

Содержание

1 Abstract	1
2 Введение	2
3 Research question	2
4 Контекст литературы	2
5 Метод	3
6 Experimental setup	3
7 Results	3
7.1 Pendulum-v1	3
7.2 HalfCheetah-v5	3
8 Discussion	4
9 Limitations	4
10 Future work	5
11 Conclusion	5
12 References and project artifacts	5

Коротко. Проект проверяет одну центральную идею из n-step SAC: помогает ли замена стандартного 1-step critic target на n-step target в условиях ограниченного compute. Итоговый результат на HalfCheetah-v5: n=3 улучшил обучение относительно baseline, а n=5 провалился при тех же hyperparameters. Главный вывод: n-step horizon - это параметр стабильности, а не ручка «чем больше, тем лучше».

1 Abstract

This project studies a compact n-step modification of Soft Actor-Critic (SAC) under a limited compute budget. The main question is whether replacing the standard 1-step critic target with n-step targets improves sample efficiency in continuous-control tasks, and where the horizon becomes unstable. I implement a configurable n-step buffer

before the replay buffer and compare SAC with $n=1$, $n=3$, and $n=5$. The experiments include Pendulum-v1 as a sanity check and HalfCheetah-v5 as the main benchmark. On HalfCheetah-v5, $n=3$ reaches the best final return, while $n=5$ fails to learn a strong running policy under the same hyperparameters. The result suggests that n -step credit assignment can help SAC, but the horizon is a stability parameter rather than a simple bigger-is-better knob.

2 Введение

Soft Actor-Critic (SAC) - сильный off-policy actor-critic алгоритм для задач continuous control. В стандартной постановке critic обучается по one-step temporal-difference target. Такой target устойчивый и простой, но распространяет информацию о награде назад только на один шаг.

N -step returns предлагают более агрессивную идею: накопить несколько rewards перед bootstrap. Это может ускорить credit assignment, особенно если полезное действие влияет на reward не сразу. Но в off-policy обучении это не бесплатно: более длинный фрагмент из replay buffer мог быть собран старой политикой, что повышает off-policy bias и target variance.

Поэтому проект не пытается заявить новый SOTA-алгоритм. Вместо этого он изолирует один механизм - horizon critic target - и проверяет его на небольшом воспроизводимом эксперименте.

3 Research question

Главный вопрос проекта:

Помогают ли n -step returns алгоритму SAC при ограниченном compute, и насколько чувствителен результат к выбранному horizon?

Сравнивались три варианта:

Method	Description
SAC $n=1$	standard 1-step SAC baseline
SAC $n=3$	SAC with moderate 3-step target
SAC $n=5$	SAC with longer and riskier 5-step target

Цель - не доказать универсальную оптимальность одного horizon, а показать, проявляется ли ожидаемый trade-off в компактной и воспроизводимой постановке.

4 Контекст литературы

Проект находится между стандартным SAC и свежими идеями n -step или sequence-aware SAC.

- **SAC** - базовый off-policy actor-critic с entropy regularization, twin critics, target networks и replay buffer.
- **SACn** важен как направление, которое фокусируется на сложности n -step returns в off-policy SAC и мотивирует stabilization/correction.
- **T-SAC / Chunking the Critic** важен тем, что использует trajectory chunks и n -step targets вместе с sequence-aware critic.

Полное воспроизведение SACn или T-SAC не делалось. Вместо этого проект тестирует общий для этих направлений механизм: что происходит, когда в SAC меня-

ется n-step horizon critic target.

5 Метод

Реализация сохраняет стандартную структуру SAC:

- squashed Gaussian actor;
- twin Q-critics;
- target critic networks;
- replay buffer;
- entropy regularization and automatic alpha tuning;
- periodic deterministic evaluation.

Главное изменение - n-step buffer, который стоит перед replay buffer. Для окна transitions он строит aggregated transition:

```
reward_n = r_t + gamma r_{t+1} + ... + gamma^(n-1) r_{t+n-1}
discount_n = gamma^k
next_state = s_{t+k}
```

где $k \leq n$, если эпизод завершился раньше. Critic target после этого имеет вид:

```
target = reward_n + discount_n * target_value(next_state)
```

Для $n=1$ это сводится к обычному SAC.

Почему это хороший ablation? В экспериментах меняется только horizon critic target: $n=1$, $n=3$, $n=5$. Hyperparameters оставлены одинаковыми, чтобы проверить именно чувствительность к horizon, а не эффект ручного tuning.

6 Experimental setup

Использовались две среды:

Environment	Role	Steps	Seed
Pendulum-v1	sanity check	30,000	0
HalfCheetah-v5	main benchmark	100,000	0

Обучение выполнялось на Ryzen 5700 laptop через WSL Ubuntu, CPU training. Evaluation делался deterministic policy rollout'ами и сохранялся в CSV. В репозитории включены configs, raw logs, plots и summary table.

7 Results

7.1 Pendulum-v1

На Pendulum-v1 все три варианта успешно обучились и пришли к похожему final quality. Этот эксперимент используется как sanity check: он подтверждает, что реализация работает и n-step target construction не ломает SAC на простой среде.

7.2 HalfCheetah-v5

Главный эксперимент - ablation на HalfCheetah-v5.

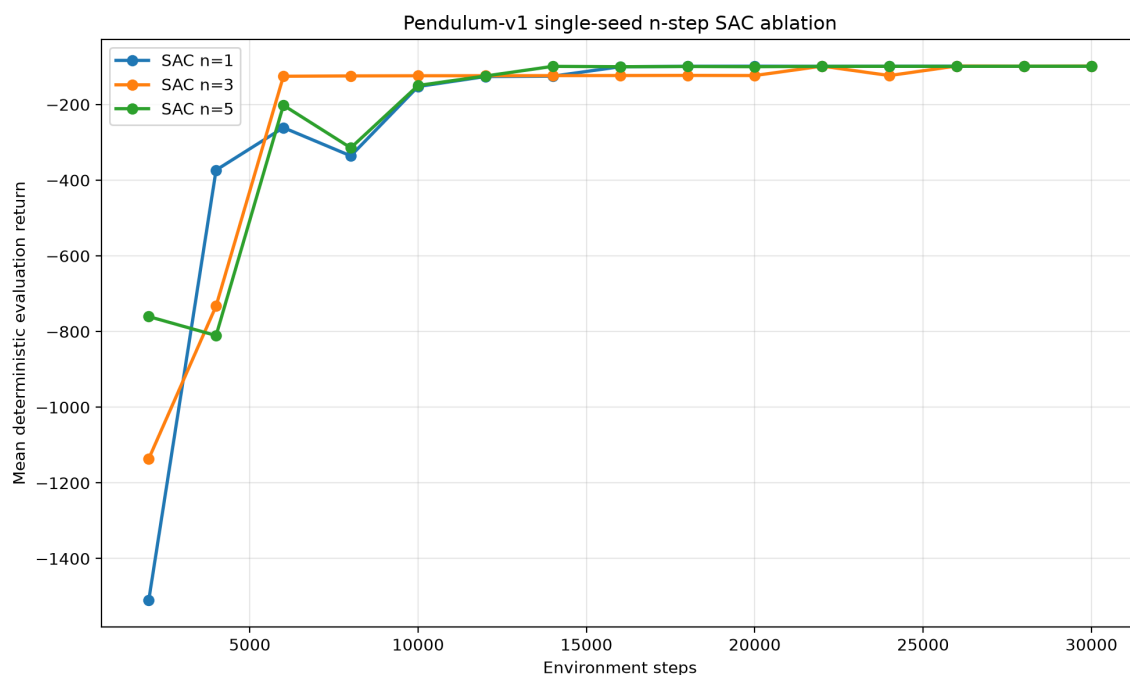


Рис. 1. Pendulum-v1 single-seed n-step SAC ablation

Method	Final eval return	Best eval return
SAC n=1	4354.49	4354.49
SAC n=3	4578.91	4578.91
SAC n=5	764.49	833.37

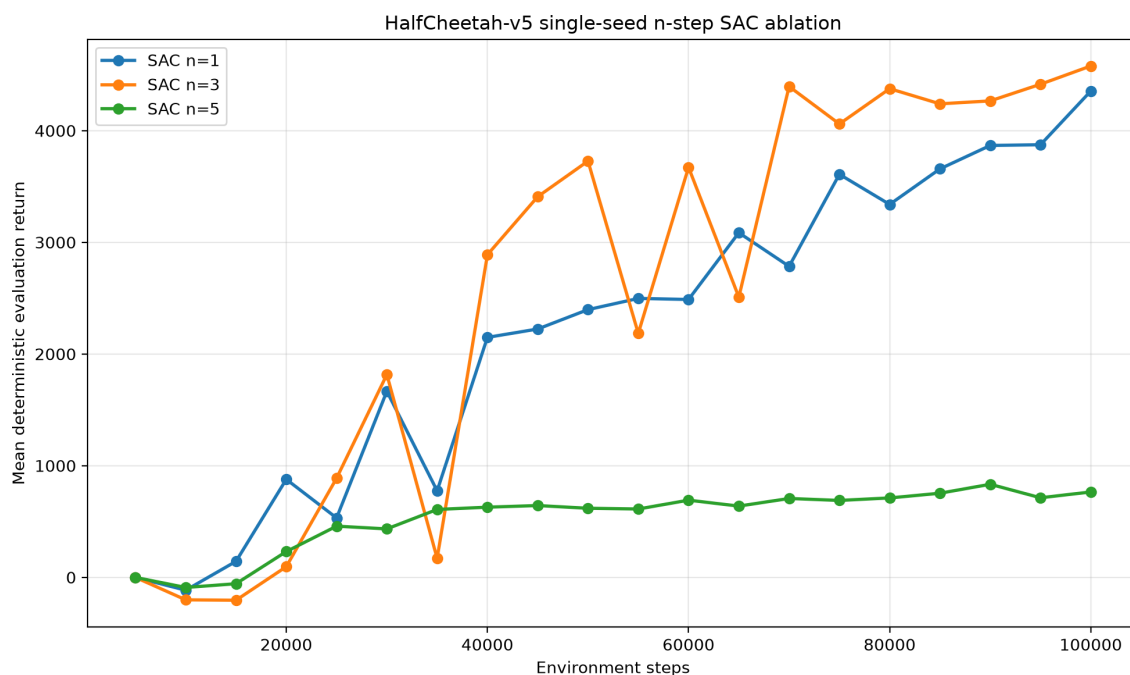


Рис. 2. HalfCheetah-v5 single-seed n-step SAC ablation

Главный результат. $n=3$ дал лучший final return и быстрее вышел к сильному поведению. $n=1$ остался сильным baseline. $n=5$ не научился устойчивому running policy при тех же hyperparameters.

Другие ограничения:

- только две среды;
- нет полного SACn correction mechanism;
- нет transformer или sequence-aware critic;
- нет отдельного hyperparameter tuning для каждого horizon;
- ограниченный CPU compute budget.

Корректная формулировка вывода: в этом limited-compute single-seed ablation, 3-step SAC улучшил HalfCheetah learning относительно 1-step baseline, а 5-step SAC провалился при тех же hyperparameters.

10 Future work

Наиболее полезные продолжения:

1. запустить 3-5 seeds;
2. добавить Hopper или Walker2d;
3. посчитать confidence intervals и failure rates;
4. сравнить naive n-step SAC с SACn-style correction;
5. отдельно tune hyperparameters для каждого horizon;
6. попробовать sequence-aware critic в духе T-SAC.

11 Conclusion

Проект реализует и тестирует configurable n-step targets в SAC. Pendulum-v1 подтверждает корректность реализации, а HalfCheetah-v5 показывает основной trade-off: $n=3$ помогает в этой постановке, но $n=5$ проваливается при тех же hyperparameters.

Итоговая мысль:

N-step horizon is a stability parameter, not a bigger-is-better knob.

12 References and project artifacts

- Repository: Therad445/nstep-sac-reproducibility.
- Main implementation: `src/rl/nstep_buffer.py`, `src/rl/sac.py`, `src/train.py`.
- Experiment logs: `results/raw/*.csv`.
- Plots: `results/plots/*.png`.
- Summary table: `results/halfcheetah_summary.csv`.
- Slides: `slides/nstep_sac_template_style_deck_ru.pptx` and `slides/nstep_sac_template_`